Test report

This report aggregates all test suites in one PDF.

String utilities

clip-str

Code

```
#clip-str("Computed Depth Maps", 8)
#clip-str("Depth", 10)
```

Output

Computed
Depth

truncate-title

Code

```
#truncate-title("Very long title",
enabled: true, max-chars: 8)
#truncate-title("Very long title",
enabled: false)
#truncate-title("Long", enabled: true,
max-chars: 3)
```

Output

Very …
Very long title
Lon

fit-lines

Code

```
#fit-lines("Line 1\nLine 2", max-chars:
6, max-lines: 2)
#fit-lines("No auto wrapping", max-chars:
6, max-lines: 1)
```

Output

Line 1
Line 2
No auto wrapping

Geometry helpers

chars-cap

Code

```
#chars-cap(2.0)
#chars-cap(4.0, min: 10, scale: 5.0)
```

Output

8
20

node-size

Code

```
#let ds = make-dataset("A", images: 3,
image-size: (1.9, 2.3), image-spacing:
0.22)
#node-size(ds)
#node-size(ds, outer: true)
```

Output

(1.9, 2.3)
(2.34, 2.7399999999999998)

node-edge

Code

```
#let ds = make-dataset("A", images: 3,
image-size: (1.9, 2.3), image-spacing:
0.22)
#node-edge(ds, side: "right")
#node-edge(ds, side: "right", outer:
true)
#node-edge(ds, side: "right", outer:
true, outer-value: 0.8)
```

Output

1.95
2.39
2.75

node-anchor

Code

```
#let box = make-box("B", size: (2.4,
1.8), pos: (5, 0))
#node-anchor(box, side: "left")
#node-anchor(box, side: "top")
#node-anchor(box, side: "right", outer:
true, outer-value: 0.5)
```

Output

(3.8, 0)
(5, 0.9)
(6.7, 0)

auto-pos-right

Code

```
#let left = make-box("Left", size: (2.2,
1.6), pos: (2, 0))
#auto-pos-right(left, 3.0, gap: 1.25)
#auto-pos-right(left, 3.0, gap: 1.25, y:
-1.0, outer-after: false)
```

Output

(5.85, 0)
(5.85, -1.0)

side-dir

Code

```
#side-dir("left")
#side-dir("right")
#side-dir("top")
#side-dir("bottom")
```

Output

(-1.0, 0.0)
(1.0, 0.0)
(0.0, 1.0)
(0.0, -1.0)

Node constructors

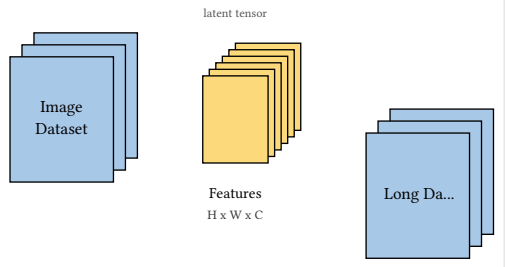
make-dataset

Code

```
#graph-canvas({
  let ds1 = make-dataset(
    "Image\nDataset",
    images: 3,
    image-size: (1.9, 2.3),
    image-spacing: 0.22,
    pos: (1.0, 0.0),
  )
  let ds2 = make-dataset(
    "Features",
    subtitle: "H x W x C",
    legend: "latent tensor",
    legend-position: "top",
    images: 6,
    image-size: (1.2, 1.6),
    image-spacing: 0.12,
    title-position: "below",
    after: ds1,
    gap: 1.2,
    color: rgb("#fcd97a"),
    title-size: 0.52em,
    subtitle-size: 0.44em,
    legend-size: 0.42em,
    wrap-lines: 2,
  )
  let ds3 = make-dataset(
    "Long Dataset Title",
    title-truncate: true,
    max-title-chars: 10,
    after: ds2,
    gap: 1.2,
    y: -1.4,
  )

  draw-node(ds1)
  draw-node(ds2)
  draw-node(ds3)
})
```

Output



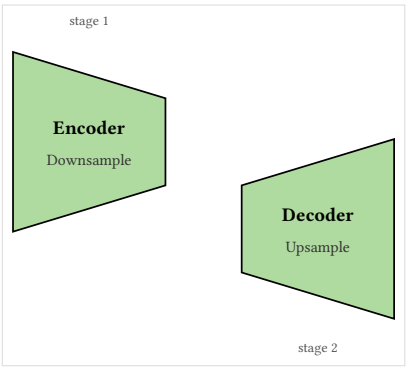
make-trapezoid

Code

Output

```
#graph-canvas({
  let enc = make-trapezoid(
    "Encoder",
    subtitle: "Downsample",
    legend: "stage 1",
    legend-position: "top",
    mode: "encoder",
    width: 2.8,
    big-half: 1.65,
    small-half: 0.80,
    pos: (2.0, 0.0),
    color: rgb("#b0dba0"),
    title-size: 0.66em,
    subtitle-size: 0.54em,
    legend-size: 0.48em,
    wrap-lines: 2,
  )
  let dec = make-trapezoid(
    "Decoder",
    subtitle: "Upsample",
    legend: "stage 2",
    legend-position: "below",
    mode: "decoder",
    width: 2.8,
    big-half: 1.65,
    small-half: 0.80,
    after: enc,
    gap: 1.4,
    y: -1.6,
    color: rgb("#b0dba0"),
  )

  draw-node(enc)
  draw-node(dec)
})
```

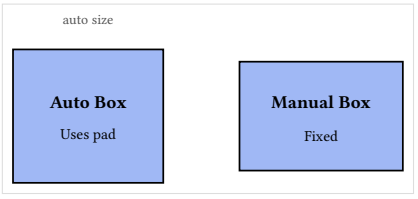


make-box

Code

```
#graph-canvas({
  let auto_box = make-box(
    "Auto Box",
    subtitle: "Uses pad",
    legend: "auto size",
    legend-position: "top",
    pad-x: 0.7,
    min-w: 2.6,
    min-h: 1.8,
    pos: (2.0, 0.0),
  )
  let manual = make-box(
    "Manual Box",
    subtitle: "Fixed",
    size: (3.0, 2.0),
    after: auto_box,
    gap: 1.4,
    color: rgb("#a0b8f5"),
```

Output



```
    wrap-lines: 2,
  )

  draw-node(auto_box)
  draw-node(manual)
})
```

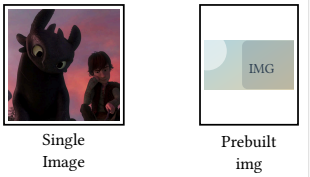
make-image-node

Code

```
#graph-canvas({
  let from_src = make-image-node(
    "Single\nImage",
    src: "/assets/test.jpg",
    image-width: 2.2,
    image-height: 2.2,
    image-pad: 0.08,
    unit: 0.72cm,
    title-position: "below",
    pos: (2.0, 0.0),
  )
  let from_img = make-image-node(
    "Prebuilt\nimg",
    img: image("/assets/sample.svg",
width: 5cm),
    image-size: (1.8, 2.2),
    image-pad: 0.08,
    unit: 0.72cm,
    title-position: "below",
    after: from_src,
    gap: 1.4,
  )

  draw-node(from_src)
  draw-node(from_img)
})
```

Output



make-image-dataset

Code

```
#graph-canvas({
  let ds = make-image-dataset(
    "Image\nDataset",
    src: "/assets/test.jpg",
    images: 4,
    image-width: 1.6,
    image-height: 2.1,
    image-spacing: 0.16,
    image-pad: 0.08,
    unit: 0.72cm,
    title-position: "below",
    pos: (2.0, 0.0),
  )

  draw-node(ds)
})
```

Output

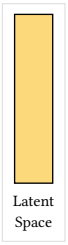
make-latent-space

Code

```
#graph-canvas({
  let latent = make-latent-space(
    "Latent\nSpace",
    height: 3.1,
    pos: (2.0, 0.0),
  )

  draw-node(latent)
})
```

Output



Arrow routing

make-arrow and draw-arrow

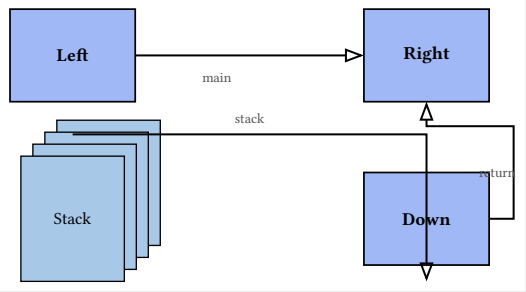
Code

```
#graph-canvas({
  let left = make-box("Left", pos: (1.5,
0.8))
  let right = make-box("Right", pos:
(8.0, 0.8))
  let down = make-box("Down", pos: (8.0,
-2.2))
  let stack = make-dataset("Stack",
images: 4, pos: (1.5, -2.2))

  draw-node(left)
  draw-node(right)
  draw-node(down)
  draw-node(stack)

  let a = make-arrow(
    left,
    right,
    out-side: "right",
    in-side: "left",
    auto-spacing: true,
    spacing: 0.6,
    mode: "hvh",
    mode-shift: (2, 1.4),
    label: [main],
    label-side: "below",
    label-gap: 0.35,
    label-dx: 0.15,
    label-dy: -0.05,
  )
  let b = make-arrow(
    stack,
    down,
    out-side: "top",
    in-side: "bottom",
    from-outer: true,
    from-outer-value: 0.4,
    to-outer: true,
    to-outer-value: 0.3,
    auto-spacing: false,
    mode: "hv",
    mode-shifts: (1.2, 0.8),
    label: [stack],
  )
  let c = make-arrow(
    down,
    right,
    out-side: "right",
    in-side: "bottom",
    mode: "vh",
    label: [return],
  )
```

Output



```
  draw-arrow(a)
  draw-arrow(b)
  draw-arrow(c)
})
```

edge-label

Code

```
#graph-canvas({
  draw.line((1, 0), (6, 0), mark: (end:
">"), stroke: (paint: black, thickness:
0.75pt))
  edge-label((1, 0), (6, 0), [input],
side: "above", gap: 0.30, dx: 0.2, dy:
0.0, size: 0.5em)
})
```

Output

draw-arrows side distribution

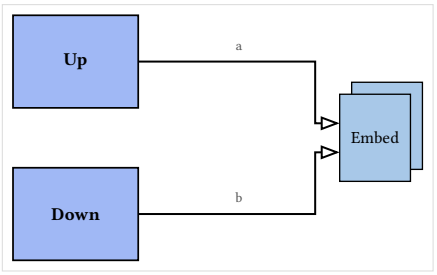
Code

```
#graph-canvas({
  let up = make-box("Up", pos: (1.5,
1.4))
  let down = make-box("Down", pos: (1.5,
-1.4))
  let target = make-dataset("Embed",
images: 2, image-size: (1.3, 1.6), pos:
(7.0, 0.0))

  draw-node(up)
  draw-node(down)
  draw-node(target)

  let a = make-arrow(up, target, out-
side: "right", in-side: "left", label:
[a])
  let b = make-arrow(down, target, out-
side: "right", in-side: "left", label:
[b])
  draw-arrows((a, b))
})
```

Output

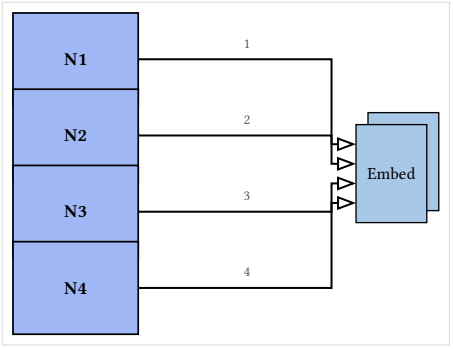


draw-arrows side distribution (4 arrows)

Code

```
#graph-canvas({
  let n1 = make-box("N1", pos: (1.2,
2.1))
  let n2 = make-box("N2", pos: (1.2,
0.7))
  let n3 = make-box("N3", pos: (1.2,
-0.7))
  let n4 = make-box("N4", pos: (1.2,
-2.1))
  let target = make-dataset("Embed",
images: 2, image-size: (1.3, 1.8), pos:
(7.0, 0.0))
```

Output



```
  draw-node(n1)
  draw-node(n2)
  draw-node(n3)
  draw-node(n4)
  draw-node(target)

  let arrows = (
    make-arrow(n1, target, out-side:
"right", in-side: "left", label: [1]),
    make-arrow(n2, target, out-side:
"right", in-side: "left", label: [2]),
    make-arrow(n3, target, out-side:
"right", in-side: "left", label: [3]),
    make-arrow(n4, target, out-side:
"right", in-side: "left", label: [4]),
  )
  draw-arrows(arrows)
})
```

draw-arrows with auto-distribute false

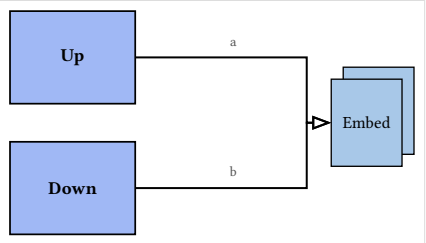
Code

```
#graph-canvas({
  let up = make-box("Up", pos: (1.6,
1.2))
  let down = make-box("Down", pos: (1.6,
-1.2))
  let target = make-dataset("Embed",
images: 2, image-size: (1.3, 1.6), pos:
(7.0, 0.0))

  draw-node(up)
  draw-node(down)
  draw-node(target)

  let arrows = (
    make-arrow(up, target, out-side:
"right", in-side: "left", label: [a]),
    make-arrow(down, target, out-side:
"right", in-side: "left", label: [b]),
  )
  draw-arrows(arrows, auto-distribute:
false)
})
```

Output



Defaults

constructor defaults (core nodes + arrows)

Code

```
#let make-dataset = set-dataset-
defaults(options: (
  images: 3,
  image-size: (1.4, 1.8),
  image-spacing: 0.14,
  title-position: "below",
))
#let make-trapezoid = set-trapezoid-
defaults(options: (
  width: 2.2,
  big-half: 1.25,
  small-half: 0.65,
))
#let make-box = set-box-defaults(options:
(
  min-w: 2.1,
  min-h: 1.5,
  pad-x: 0.55,
))
#let make-arrow = set-arrow-
defaults(options: (
  spacing: 0.48,
  label-gap: 0.32,
))

#graph-canvas(length: 0.58cm, {
  let ds = make-dataset("Input", pos:
(1.0, 0.0))
  let enc = make-trapezoid("Encoder",
subtitle: "Conv", after: ds, gap: 0.9)
  let head = make-box("Head", subtitle:
"MLP", after: enc, gap: 0.9)
  let aux = make-dataset("Aux", images:
2, image-size: (1.0, 1.3), pos: (5.2,
1.7))

  draw-node(ds)
  draw-node(enc)
  draw-node(head)
  draw-node(aux)

  let arrows = (
    make-arrow(ds, enc, from-outer: true,
label: [input]),
    make-arrow(enc, head, label:
[features]),
    make-arrow(aux, head, out-side:
"right", in-side: "top", from-outer:
true, mode: "hv", label: [aux]),
  )
  draw-arrows(arrows)
})
```

Output

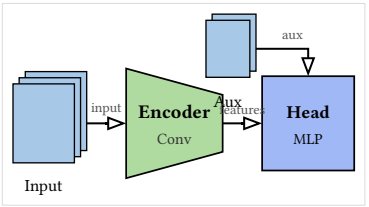


image + latent defaults

Code

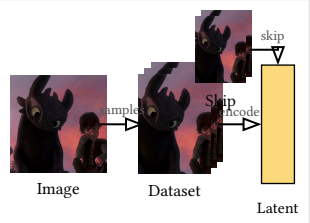
```
#let make-image-node = set-image-node-
defaults(options: (
  src: "/assets/test.jpg",
  image-width: 1.9,
  image-height: 1.9,
  image-pad: 0.06,
  title-position: "below",
))
#let make-image-dataset = set-image-
dataset-defaults(options: (
  src: "/assets/test.jpg",
  images: 3,
  image-width: 1.4,
  image-height: 1.9,
  image-spacing: 0.14,
  image-pad: 0.04,
  title-position: "below",
))
#let make-latent-space = set-latent-
space-defaults(options: (
  width: 0.75,
  height: 2.7,
  title-position: "below",
))
#let make-arrow = set-arrow-
defaults(options: (
  spacing: 0.42,
  label-gap: 0.30,
))

#graph-canvas(length: 0.58cm, {
  let img = make-image-node("Image", pos:
(1.0, 0.0))
  let ds = make-image-dataset("Dataset",
after: img, gap: 1.0)
  let latent = make-latent-
space("Latent", after: ds, gap: 1.0)
  let skip = make-image-dataset("Skip",
images: 2, image-width: 1.0, image-
height: 1.3, pos: (4.7, 1.7))

  draw-node(img)
  draw-node(ds)
  draw-node(latent)
  draw-node(skip)

  let arrows = (
    make-arrow(img, ds, label:
[samples]),
    make-arrow(ds, latent, from-outer:
true, label: [encode]),
    make-arrow(skip, latent, out-side:
"right", in-side: "top", from-outer:
true, mode: "hv", label: [skip]),
```

Output



```
  )
  draw-arrows(arrows)
})
```

Emoji

draw-node-emoji

Code

```
#graph-canvas({
  let lock = make-box("Secure Box", pos:
(4.0, 0.0))
  draw-node(lock)

  draw-node-emoji(
    lock,
    kind: "lock-closed",
    place: "top",
    gap: 0.6,
    shift: (0.2, 0.0),
    size: 1.0em,
    color: rgb("#333333"),
    use-outer: false,
    key-state: "encrypted",
    state-text: "enc",
    state-size: 0.38em,
    state-gap: 0.25,
    state-position: "above",
  )

  draw-node-emoji(
    lock,
    kind: "key",
    place: "right",
    key-state: "decrypted",
  )

  draw-node-emoji(
    lock,
    kind: "custom",
    emoji: [X],
    place: "under",
  )
})
```

Output